

UNIVERSITÉ DE KINSHASA
FACULTÉ POLYTECHNIQUE
DÉPARTEMENT DE GÉNIE ÉLECTRIQUE



Travail de fin de cycle :

**CONCEPTION ET RÉALISATION D'UN SYSTÈME ÉLECTRONIQUE DE
CONTRÔLE VOCAL D'UN MOTEUR DC À AIMANT PERMANENT**

Travail réalisé par

ROGER BOSOLINDO EDHIENGENE

sous la direction de

Année 2025-2026

Remerciements

Table des matières

Remerciements	2
Table des matières	3
Table des figures	5
Liste des tableaux	6
Liste des sigles et abréviations	7
1 Introduction	8
2 Présentation du microcontrôleur Arduino Nano 33 BLE SENSE REV2	10
2.1 Introduction aux Microcontrôleurs	10
2.2 Types de Microcontrôleurs	10
2.3 Arduino Nano 33 BLE Sense Rev2	11
2.3.1 Le Microphone Micro-Electro-Mechanical Systems (Systèmes Micro-Électro-Mécaniques) (MEMS) HP34DT05-A	12
3 Conception du modèle deep learning de reconnaissance vocale	13
3.1 Machine Learning	13
3.1.1 Définition (Entraînement)	14
3.2 Types de Problèmes que Nous Poumons Résoudre Grâce au Machine Learning (apprentissage automatique) (ML)	14
3.2.1 Apprentissage Supervisé	14
3.2.2 Apprentissage Non Supervisé	14
3.2.3 Apprentissage Semi-Supervisé	15
3.2.4 Apprentissage par Renforcement	15
3.2.5 Définition (Apprentissage Automatique, Tom Mitchell, 1997)	15
3.3 Optimisation	15
3.3.1 Régression Linéaire	15
3.3.2 La Descente de Gradient	15
3.4 Courbes d'Apprentissage	16
3.4.1 Définition (Généralisation)	17
3.4.2 Définition (Sur-Apprentissage)	17
3.4.3 Définition (Sous-Apprentissage)	17
3.5 Régularisation	17
3.5.1 Régularisation Ridge	17
3.5.2 Régularisation Lasso	17
3.6 Introduction aux Réseaux de Neurones Artificiels avec Keras	17
3.6.1 Le Perceptron	18

3.6.2	La Rétropropagation	18
3.6.3	Keras	18
4	Analyse du moteur DC à aimant permanent	19
5	Conception du pont en H de commande du moteur avec Arduino	20
6	Présentation des résultats des tests du système	21
7	Contraintes et possibilité d'étendre le projet	22
8	Conclusion	23
A	Modules Python utilisés	24
B	Codes du projet	25
	Bibliographie	26

Table des figures

Liste des tableaux

Liste des sigles et abréviations

AIoT Artificial Intelligence of Things

ML Machine Learning (apprentissage automatique)

TinyML Tiny Machine Learning

DNN Deep Neural Network (réseau de neurones profond)

NN Neural Network (réseau de neurones)

ULS Unité Linéaire à Seuil

ReLU Rectified Linear Unit

SGD Stochastic Gradient Descent (descente de gradient stochastique)

MSE Mean Squared Error (erreur quadratique moyenne)

RMSE Root Mean Squared Error

API Application Programming Interface

DC Direct Current (courant continu)

PWM Pulse Width Modulation (modulation de largeur d'impulsion)

BLE Bluetooth Low Energy

NFC Near Field Communication

ADC Analog-to-Digital Converter (convertisseur analogique-numérique)

GPIO General Purpose Input/Output

CPU Central Processing Unit (unité centrale de traitement)

RAM Random Access Memory (mémoire vive)

EEPROM Electrically Erasable Programmable Read-Only Memory

IDE Integrated Development Environment (environnement de développement intégré)

RISC Reduced Instruction Set Computer

CISC Complex Instruction Set Computer

FPGA Field Programmable Gate Array

PIC Peripheral Interface Controller

ARM Advanced RISC Machines

FPU Floating Point Unit (unité de calcul en virgule flottante)

MEMS Micro-Electro-Mechanical Systems (microsystèmes électromécaniques)

IMU Inertial Measurement Unit (centrale inertielle)

Chapitre 1

Introduction

Pour mon travail de fin de cycle sanctionnant la fin de mon parcours de licence en sciences de l'ingénieur, nous avons investi sur un sujet d'actualité ; un sujet qui rassemble l'électronique embarquée, l'informatique, les mathématiques ainsi que l'étude des signaux et systèmes. Des disciplines que nous avons explorées tout au long de ce voyage, même si certaines comme l'électronique n'ont été que récemment assimilées.

Nous parlerons d'un système électronique de contrôle vocal d'un moteur à courant continu à aimant permanent. C'est un projet Artificial Intelligence of Things (Intelligence Artificielle des Objets) ([AIoT](#)), dans lequel nous utiliserons un modèle d'apprentissage automatique conçu à partir d'un réseau de neurones, qui aura pour mission d'identifier des mots clés afin de piloter notre petit moteur.

Un tel système peut être scindé en ces sous-systèmes :

- le modèle d'apprentissage automatique ;
- la chaîne d'acquisition et de traitement de la voix, qui comprend le microcontrôleur Arduino ;
- le driver de commande qui relie les systèmes précédents au moteur ;
- le moteur DC à aimant permanent ;
- le système d'alimentation du système total.

Dans la suite, après avoir analysé tous ces sous-systèmes dans les chapitres correspondants, nous présenterons le prototype sur un circuit imprimé réalisé sur Eagle, Ki-Cad ou SimulIDE en fonction du logiciel disponible.

Il faut également ajouter que nous présentons, en annexe, un ensemble de programmes Python et Arduino que nous avons utilisés dans cette conception.

Outils logiciels

À part les trois logiciels cités ci-haut, nous avons utilisé, pour la réalisation du réseau de neurones et l'entraînement du modèle d'apprentissage automatique, les frameworks TensorFlow et Keras disponibles comme modules sous Python. Toujours avec Python, d'autres modules comme librosa, sklearn, matplotlib et numpy nous ont été très utiles pour l'acquisition et le prétraitement des données d'entraînement, de test et de validation. D'autres logiciels aussi comme Arduino **IDE!** (**IDE!**) et PyCharm ont servi soit pour l'édition soit pour la compilation.

Organisation du document

Les prochains chapitres sont organisés d'après les sous-systèmes évoqués ci-haut comme suit :

Chapitre 2

Présentation du microcontrôleur Arduino Nano 33 BLE Sense Rev2, pièce maîtresse du projet.

Chapitre ??

Conception du modèle deep learning de reconnaissance vocale.

Chapitre ??

Analyse du moteur DC à aimant permanent EG-530AD-9B de Mabuchi Motor.

Chapitre ??

Conception du pont en H de commande du moteur avec Arduino.

Chapitre 6

Présentation des résultats des tests du système et conception du système d'alimentation.

Chapitre 7

Contraintes, limitations et possibilités d'étendre le projet.

Chapitre ??

Conclusion.

Étant donné que nous ne pouvons pas parler de tout ce que nous avons fait dans ce projet car les exigences de la rédaction nous en empêchent, nous avons quand même prévu quelques extensions aux annexes et sur internet, à travers mon compte GitHub. Le lien du dépôt est : <https://github.com/nobohie/prog-python>.

Chapitre 2

Présentation du microcontrôleur Arduino Nano 33 BLE Sense Rev2

Une unité de microcontrôleur est essentiellement un petit ordinateur sur une puce unique. Il est conçu pour gérer des tâches spécifiques au sein d'un système embarqué sans nécessiter de système d'exploitation complexe.

Ces circuits intégrés compacts contiennent un cœur de processeur (ou plusieurs), une mémoire vive (Random Access Memory (Mémoire Vive) ([RAM](#))) et une mémoire à lecture seule programmable effaçable électriquement (Electrically Erasable Programmable Read-Only Memory ([EEPROM](#))) pour stocker les programmes personnalisés qui s'exécutent sur le microcontrôleur, même lorsque l'unité est déconnectée d'une alimentation. Les microcontrôleurs intègrent les périphériques de traitement, de mémoire et d'entrée/sortie (E/S), notamment les minuteries, les compteurs et les convertisseurs analogiques-numériques (Analog-to-Digital Converter (Convertisseur Analogique-Numérique) ([ADC](#))), dans une seule unité autonome efficace et économique.

Voici les principaux composants d'un microcontrôleur :

- **Unité centrale de traitement (Central Processing Unit (Unité Centrale de Traitement) ([CPU](#)))** : le composant principal responsable de l'exécution des instructions et du contrôle des opérations.
- **Mémoire** : une mémoire volatile ([RAM](#)) et une mémoire flash non volatile pour stocker le jeu d'instructions de programmation du microcontrôleur.
- **Périphériques** : en fonction de l'application envisagée, il peut contenir divers composants auxiliaires.

2.1 Types de microcontrôleurs

Les premiers types de microcontrôleurs sont issus des avancées réalisées dans la fabrication des microprocesseurs, lorsque les chercheurs ont développé des techniques pour intégrer des composants de [CPU](#), de mémoire et de périphériques dans des puces uniques. Gary Boone et Michael Cochran, ingénieurs chez Texas Instruments, sont connus pour avoir créé le premier microcontrôleur en 1971 [?]. Voici quelques-uns des types de microcontrôleurs les plus courants :

Microcontrôleurs 8 bits. Type de microcontrôleur le plus élémentaire, doté d'un traitement et d'une mémoire limités, généralement utilisés dans les petits appareils tels que les jouets et les télécommandes.

Microcontrôleurs 16 bits. Deux fois plus performants que les modèles 8 bits, ils sont utilisés pour des applications plus complexes : dispositifs médicaux, systèmes automobiles et systèmes de contrôle industriel.

Microcontrôleurs 32 bits. Il s'agit du type le plus puissant et le plus riche en fonctionnalités. Il est utilisé pour des applications exigeantes, telles que les consoles de jeu, les appareils de divertissement et l'automatisation industrielle haut de gamme.

Microcontrôleurs à jeu d'instructions réduit (RISC). Ils intègrent une architecture de conception qui simplifie et améliore les opérations en exécutant moins d'instructions de calcul plus rapidement que d'autres méthodologies, telles que l'architecture d'ordinateur à jeu d'instructions complexes (CISC! (CISC!)).

Microcontrôleurs ARM. Ces types de microcontrôleurs intègrent l'architecture Advanced RISC Machine (ARM), y compris le sous-ensemble moderne ARM Cortex, qui renforce les performances et la fiabilité. Ils sont très utilisés dans les appareils mobiles, les systèmes automobiles et les systèmes de contrôle industriels.

Microcontrôleurs PIC. Développé par Microchip Technology, le microcontrôleur PIC! (PIC!) est le plus petit microcontrôleur au monde. On le trouve dans la robotique, l'automatisation domestique et industrielle et les systèmes d'énergies renouvelables.

Microcontrôleurs basés sur FPGA. Couramment utilisés dans les applications qui nécessitent le traitement de signaux numériques, le traitement vidéo et les réseaux haut débit, ils utilisent des réseaux de portes programmables in situ (Field-Programmable Gate Array (FPGA)), qui peuvent être configurés et reconfigurés au niveau matériel pour obtenir des résultats de traitement hautement polyvalents et personnalisables.

Dans le cadre de ce projet, nous avons utilisé le microcontrôleur Arduino Nano 33 BLE Sense Rev2. Il s'agit d'un microcontrôleur de type ARM que nous présentons dans la prochaine section.

2.2 Arduino Nano 33 BLE Sense Rev2

Arduino est une plateforme open-source de prototypage électronique fondée en 2005 au Politecnico di Ivrea en Italie [?]. La carte Arduino intègre un microcontrôleur programmable, accessible via un IDE! simplifié basé sur le langage C/C++.

L'Arduino Nano 33 BLE Sense est une carte de développement de format compact (45 × 18 mm). Elle se distingue de la version « Nano 33 BLE » de base par l'ajout de neuf capteurs intégrés dont un microphone numérique.

Caractéristiques techniques

Architecture CPU : ARM Cortex-M4F (32 bits avec Floating Point Unit (Unité de Calcul en Virgule Flottante) (FPU)). Il s'agit d'un processeur basse consommation caractérisé par un faible nombre de petits logiciels, une faible latence et une unité de calcul en virgule flottante intégrée. L'architecture RISC 32 bits est un membre de la famille ARM Cortex-M et utilise un jeu d'instructions simplifié qui traite les instructions à 32 bits [?].

Fréquence d'horloge : 64 MHz. La fréquence d'horloge dans un microcontrôleur indique deux choses : la vitesse d'exécution et la synchronisation. Plus la fréquence est élevée, plus la vitesse d'exécution et la synchronisation des composants internes sont rapides. Tous les composants internes font des programmes et s'exécutent à un cadencement de l'horloge.

Mémoire Flash : 1 MB.

Mémoire RAM : 256 KB. Ce modèle veille en permettant aux données d'être maintenues en mémoire. Il n'est pas nécessairement que le modèle maintenant soit un modèle puissant qui la veut : on peut changer en mémoire.

Connectivité radio : Bluetooth 5.0, Bluetooth Low Energy ([BLE](#)), **NFC!** (**NFC!**), 802.15.4.

Nombre de broches [GPIO](#) : 45. Ce sont des broches à usage général ; leurs fonctions ne sont pas décidées par le matériel logiciel ; elles doivent être décidées une à une ou laisser entrer un signal.

Tensions de fonctionnement : 3.3 V et 5 V.

ADC : 18 bits, disponible sur plusieurs broches, jusqu'à 3 canaux.

PWM : disponible à la résolution.

Capteurs embarqués : La force de cette carte réside dans ses neuf capteurs intégrés :

- Gyroscope 3D et magnétomètre 3D (Inertial Measurement Unit (Unité de Mesure Inertielle) ([IMU](#)) 9 axes, accéléromètre 3D) ;
- capteurs de température et d'humidité ;
- capteur de pression barométrique, capteurs de proximité ;
- microphone numérique HP34DT05-A.

PWM (Pulse Width Modulation)

Le Pulse Width Modulation (Modulation de Largeur d'Impulsion) ([PWM](#)) est une technique numérique qui permet de simuler un signal analogique à partir d'une sortie numérique. Au lieu de mettre une sortie à 3.3 V en continu pour faire briller une LED à fond par exemple, le [PWM](#) va allumer et éteindre cette LED très rapidement. Le signal carré généré à une fréquence fixe donne l'illusion d'un niveau analogique en faisant varier le rapport cyclique : le pourcentage du temps où le signal est à l'état haut (par rapport à la période).

Le microphone HP34DT05-A

Un microphone [MEMS](#) est fabriqué par des procédés de micro-fabrication semi-conducteurs. La membrane mécanique extrêmement fine est gravée dans une membrane constituée de silicium ; les variations de pression font varier la capacité électrique qui est ensuite convertie en signal numérique. La fréquence d'échantillonnage peut aller jusqu'à 3.95 MHz [?].

Le HP34DT05-A présente une réponse fréquentielle plate de 100 Hz à 10 kHz avec une légère atténuation en dessous de 100 Hz et une coupure progressive au-delà de 10 kHz.

Grâce au microphone numérique intégré à l'Arduino que nous utilisons, toutes les manœuvres de filtrage analogique (Butterworth, Tchebychev) et la chaîne d'acquisition ne sont plus à considérer.

Chapitre 3

Conception du modèle deep learning de reconnaissance vocale

Notre objectif peut à priori se réduire au fonctionnement d'une forme d'intelligence et d'émergence dans le domaine du microcontrôleur. Pour atteindre ce but, nous nous pencherons vers le domaine de l'intelligence artificielle et de l'émergence, plus connu comme le « *Machine Learning* ».

Étant donné que nous travaillons sur un petit modèle capable de fonctionner sur un microcontrôleur tel que Arduino, nous irons vers le « *Tiny Machine Learning* » (Tiny Machine Learning ([TinyML](#)) en abrégé). C'est un ensemble des techniques permettant de fonctionner et d'exécuter des algorithmes ML sur des microcontrôleurs [?].

Les motifs qui nous poussent à choisir cette approche sont :

- **Autonomie** : l'absence de connexion internet, nous n'en aurons pas besoin pour faire tourner le système ;
- **Latence** : de l'ordre de quelques millisecondes ;
- **Consommation** : vraiment infime en comparaison avec des modèles hébergés dans des clouds consommant des kilowatts ;
- **Coût matériel** : faible, quelques dizaines de dollars ;
- **Confidentialité** : les données ne quittent jamais l'appareil ou le microcontrôleur.

3.1 Apprentissage automatique (*Machine Learning*)

3.1.1 Définitions et types de problèmes

Définition 1 (Apprentissage automatique [?, p. 2]). Un système ML passe en général par deux phases :

1. **Entraînement** : le système apprend une tâche T et améliore une mesure de performance P à partir d'une expérience E ; il améliore progressivement la fonction de coût jusqu'à ce qu'elle converge vers un minimum.
2. **Inférence** : il applique ce qu'il a appris sur de nouvelles données et effectue ses prédictions.

Quatre types de problèmes que nous pouvons résoudre grâce au [ML](#) :

1. l'apprentissage supervisé ;
2. l'apprentissage non supervisé ;
3. l'apprentissage semi-supervisé ;
4. l'apprentissage par renforcement.

Apprentissage supervisé

Définition 2 (Apprentissage supervisé [?]). On suppose qu'on dispose d'un jeu de m observations $\{(x^i, y^i)\}_{i=1}^m$ dans un espace $X \times Y$. En apprentissage supervisé, on cherche une fonction $\phi : X \rightarrow Y$ fixe et inconnue telle que les fonctions ϕ obtenues à partir des observations d'apprentissage approchent au mieux cette fonction ϕ .

Notons $\{x^i\}_{i=1, \dots, m}$ les observations dans un espace X et $\{y^i\}$ les étiquettes dans un espace Y .

Classification. Pour un problème de classification multi-classe (avec C catégories) :

$$f(x) = \arg \max_{c \in \{1, \dots, C\}} g_c(x)$$

où $g_c : X \rightarrow \mathbb{R}$ est la c -ième *fonction de décision*.

Définition 3 (Frontière et régions de décision). Dans le cas d'une classification multi-classe, les régions de décision sont :

$$R_c = \{x \in X \mid g_c(x) = \max_k g_k(x)\}.$$

Définition 4 (Fonction de décision (classification binaire)). On appelle *fonction de décision* la fonction $g : X \rightarrow \mathbb{R}$ telle que :

$$f(x) = \begin{cases} 0 & \text{si } g(x) \leq 0 \\ 1 & \text{si } g(x) > 0 \end{cases}$$

Définition 5 (Une-contre-toutes (*one-vs-rest*)). L'approche consiste à entraîner un classifieur binaire pour chaque classe c , en utilisant les exemples de la classe c comme exemples positifs et toutes les autres comme exemples négatifs.

Risque empirique.

Définition 6 (Risque empirique). Soit L une fonction de perte et h une hypothèse. Le *risque empirique* est :

$$R_n(h) = \frac{1}{m} \sum_{i=1}^m L(h(x^i), y^i).$$

Définition 7 (Risque quadratique empirique ([RMSE](#))).

$$\text{RMSE}(X, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m [h(x^{(i)}) - y^{(i)}]^2}.$$

Apprentissage non supervisé

Définition 8 (Apprentissage non supervisé). On dispose d'un jeu de données $\{x^i\}_{i=1}^m$ non étiquetées dans un espace X ; il s'agit d'apprendre une certaine structure de cet espace. Les principaux exemples sont :

- le *clustering* (partitionnement) – identifier des groupes similaires dans les données ;
- la réduction de dimension – projection $X \rightarrow Z$ où $\dim Z < \dim X$.

Apprentissage semi-supervisé

Définition 9 (Apprentissage semi-supervisé). Il consiste à apprendre à partir de données partiellement étiquetées : un ensemble de données étiquetées et un ensemble de données non étiquetées, partiellement.

Apprentissage par renforcement

Définition 10 (Apprentissage par renforcement). Le système (*agent*) interagit avec un environnement et accomplit des actions, qui obtient un bon ou mauvais retour. Les applications principales sont les jeux (*échecs*, *go*, etc.) et la robotique.

3.1.2 Régression linéaire

Un modèle linéaire effectue une prédiction en calculant une somme pondérée des variables d'entrée, en y ajoutant un terme constant :

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_p x_p = \boldsymbol{\theta}^T \cdot \mathbf{x}$$

où θ_0 est le biais et θ_j les paramètres du modèle. On représente le jeu de données par une matrice de conception $X \in \mathbb{R}^{m \times p}$:

$$X = \begin{pmatrix} x_1^{(1)} & \dots & x_p^{(1)} \\ \vdots & & \vdots \\ x_1^{(m)} & \dots & x_p^{(m)} \end{pmatrix}$$

et les prédictions sont $\hat{\mathbf{y}} = X\boldsymbol{\theta}$.

Équation normale. La valeur de $\boldsymbol{\theta}$ qui minimise le Root Mean Square Error (Erreur Quadratique Moyenne) ([RMSE](#)) admet une solution analytique :

$$\hat{\boldsymbol{\theta}} = (X^T X)^{-1} X^T \mathbf{y} \quad (3.1)$$

L'équation (??) est appelée *équation normale*. On peut remarquer que cette formule fait tourner une opération matricielle de complexité $\mathcal{O}(p^2)$; elle devient coûteuse pour un grand nombre de variables.

3.1.3 Descente de gradient

Définition 11 (Fonction de coût [MSE](#)).

$$\text{MSE}(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m (\boldsymbol{\theta}^T x^{(i)} - y^{(i)})^2$$

Proposition 1 (Gradient de la [MSE](#)).

$$\nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) = \frac{2}{m} X^T (X\boldsymbol{\theta} - \mathbf{y})$$

La mise à jour des paramètres à chaque itération :

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta})$$

où η est le *learning rate* (taux d'apprentissage), un hyperparamètre crucial : trop petit, l'algorithme converge lentement ; trop grand, il risque de ne pas converger.

Types de descente de gradient.

Batch gradient descent (ordinaire) : le gradient est calculé sur l'ensemble des exemples d'entraînement à chaque étape.

Descente de gradient stochastique (SGD!) : une seule observation choisie aléatoirement à chaque étape ; plus rapide mais bruité.

Mini-batch gradient descent : compromis : le gradient est calculé sur un petit lot aléatoire d'observations.

3.1.4 Courbes d'apprentissage

Les courbes d'apprentissage représentent graphiquement l'erreur pour l'évaluation du modèle en fonction des données d'entraînement et de validation au fil des époques. Elles permettent de diagnostiquer le sous-apprentissage et le sur-apprentissage.

3.1.5 Régression polynomiale

Si la complexité des données ne peut se modéliser par un modèle linéaire, on peut enrichir les entrées avec des puissances :

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_d x^d + \varepsilon$$

L'entraîner un modèle linéaire sur ces nouvelles variables constitue la régression polynomiale.

3.1.6 Généralisation, sous-apprentissage et sur-apprentissage

Définition 12 (Généralisation). La *généralisation* est la capacité d'un modèle à faire des prédictions correctes sur de nouvelles données.

Définition 13 (Sous-apprentissage (*underfitting*)). On dit d'un modèle qu'il est en *sous-apprentissage* lorsqu'il est trop simple pour capturer la structure des données : ses performances sont insatisfaisantes même sur les données d'entraînement.

Définition 14 (Sur-apprentissage (*overfitting*)). On dit d'un modèle qu'il est en *sur-apprentissage* lorsqu'il est trop complexe et s'ajuste au bruit des données d'entraînement : ses performances sont bonnes sur les données d'entraînement mais mauvaises sur les nouvelles données.

3.1.7 Régularisation

La régularisation ajoute une pénalité à la fonction de coût afin de contrôler la complexité du modèle. L'hyperparamètre α contrôle la quantité de régularisation imposée.

Régression Ridge (régularisation ℓ_2).

Définition 15 (Régression Ridge).

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \frac{\alpha}{m} \sum_{i=1}^m \theta_i^2$$

La solution analytique est :

$$\hat{\boldsymbol{\theta}} = (X^T X + \alpha A)^{-1} X^T \mathbf{y}$$

où A est la matrice identité d'ordre $(p + 1)$.

Régression Lasso (régularisation ℓ_1).

Définition 16 (Régression Lasso).

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + 2\alpha \sum_{i=1}^m |\theta_i|$$

Lasso tend à éliminer les poids des variables les moins importantes (parcimonie).

Régression Elastic Net.

Définition 17 (Régression Elastic Net). C'est un compromis entre Ridge et Lasso, avec un paramètre de mélange $\lambda \in [0, 1]$:

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \lambda \frac{\alpha}{m} \sum_i \theta_i^2 + (1 - \lambda) 2\alpha \sum_i |\theta_i|$$

Arrêt précoce (*Early Stopping*). Une autre manière de régulariser est d'interrompre l'entraînement lorsque l'erreur de validation cesse de diminuer, afin d'éviter le sur-apprentissage.

3.1.8 Régression logistique

Classification binaire

Définition 18 (Fonction sigmoïde). La régression logistique utilise la fonction sigmoïde pour produire une probabilité :

$$\sigma(t) = \frac{1}{1 + e^{-t}}, \quad \sigma(t) \in (0, 1)$$

La prédiction du modèle est :

$$\hat{p} = \sigma(\boldsymbol{\theta}^T \mathbf{x}), \quad \hat{y} = \begin{cases} 1 & \text{si } \hat{p} \geq 0,5 \\ 0 & \text{sinon} \end{cases}$$

Fonction de coût (log-loss / entropie croisée binaire).

$$c(\boldsymbol{\theta}) = \begin{cases} -\log(\hat{p}) & \text{si } y = 1 \\ -\log(1 - \hat{p}) & \text{si } y = 0 \end{cases}$$

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \sigma(\boldsymbol{\theta}^T x^{(i)}) + (1 - y^{(i)}) \log(1 - \sigma(\boldsymbol{\theta}^T x^{(i)})) \right]$$

Classification multi-classe (Softmax)

Pour avoir un classifieur à plus de deux classes, on utilise la régression Softmax :

Définition 19 (Régression Softmax). Pour chaque classe k , on calcule un score :

$$A_k(\mathbf{x}) = \boldsymbol{\Theta}^{(k)T} \mathbf{x}$$

La probabilité d'appartenance à la classe k est donnée par la fonction Softmax :

$$\sigma(A(\mathbf{x}))_k = \frac{\exp(A_k(\mathbf{x}))}{\sum_{j=1}^K \exp(A_j(\mathbf{x}))}$$

La classification est alors : $\hat{y} = \arg \max_k \sigma(A(\mathbf{x}))_k = \arg \max_k A_k(\mathbf{x})$.

Entropie croisée catégorielle.

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \hat{p}_k^{(i)}$$

$$\nabla_{\Theta^{(k)}} J(\Theta) = \frac{1}{m} \sum_{i=1}^m (\hat{p}_k^{(i)} - y_k^{(i)}) \mathbf{x}^{(i)}$$

3.2 Réseaux de neurones artificiels

3.2.1 Introduction aux réseaux de neurones

Les réseaux de neurones artificiels existaient déjà il y a bien longtemps. Ils ont été décrits pour la première fois en 1943 par le neuro-physiologiste Warren McCulloch et le mathématicien Walter Pitts [?]. L'intérêt pour ces méthodes a connu un regain dans les années 1980 avec l'avènement du connexionnisme et de nouvelles architectures.

3.2.2 Le perceptron

Inventé en 1957 par Frank Rosenblatt, le perceptron est une architecture de nœud de neurone artificiel basée sur un seul neurone appelé « unité linéaire à seuil » (**ULS!** (**ULS!**)).

La sortie d'une couche ULS est calculée comme suit :

$$h_{W,b}(\mathbf{x}) = \phi(\mathbf{x} \cdot \mathbf{W} + b) \quad (3.2)$$

avec $\mathbf{x}$ la matrice des observations, $\mathbf{W}$ la matrice des poids, b le vecteur de biais (*bias*), ϕ la fonction d'activation.

Définition 20 (Règle d'apprentissage du perceptron).

$$W_{i,j} \leftarrow W_{i,j} + \eta (y_i - \hat{y}_i) \cdot x_i$$

où η est le taux d'apprentissage.

3.2.3 Réseau de neurones multicouche (DNN)

Un perceptron multicouche (**DNN!** (**DNN!**)) est constitué d'une couche d'entrée, d'une ou plusieurs « couches cachées » d'unités ULS, et d'une couche de sortie. L'entraînement se fait par la méthode de rétropropagation ; le changement d'un modèle ne peut pas les lier : l'adaptation, les couches de normalisation et les couches de déclarations et de sous-classement.

3.2.4 La rétropropagation

Voici les étapes de l'algorithme de rétropropagation [?] :

1. Il traite un mini-lot à la fois (par exemple 39 instances) et répète le traitement complet plusieurs fois (*époques*).
2. Chaque mini-lot est transmis à la première couche du réseau. L'algorithme calcule la sortie de chaque couche ; le résultat est transmis à la couche suivante.
3. Il mesure ensuite l'erreur du réseau à la dernière couche de sortie.
4. Il procède ensuite en sens inverse (*passé en arrière*) : en appliquant la règle de la chaîne, il mesure la contribution de chaque connexion à l'erreur.
5. Enfin, il effectue une descente de gradient pour ajuster les poids du réseau.

3.2.5 Interface de programmation Keras

Keras est un *framework* de haut niveau de TensorFlow pour le *deep learning* [?]. Il permet la construction, la compilation, l'entraînement, l'évaluation et l'exécution de modèles réseaux.

API Séquentielle

Listing 3.1 – Modèle séquentiel – compilation et entraînement

```

1 import tensorflow as tf
2
3 # Construction d'un modele sequentiel
4 model = tf.keras.Sequential([
5     tf.keras.layers.Normalization(),
6     tf.keras.layers.Dense(50, activation='relu'),
7     tf.keras.layers.Dense(50, activation='relu'),
8     tf.keras.layers.Dense(1)
9 ])
10
11 optimigen = tf.keras.optimizers.Adam(learning_rate=1e-3)
12 model.compile(loss="mse",
13               optimizer=optimigen,
14               metrics=["RootMeanSquaredError"])
15
16 history = model.fit(X_train, y_train,
17                    validation_data=(X_val, y_val),
18                    epochs=90)

```

API Fonctionnelle – Modèle Wide and Deep

Listing 3.2 – Modèle Wide and Deep avec l'API fonctionnelle Keras

```

1 import tensorflow as tf
2
3 class WideAndDeepModel(tf.keras.Model):
4     def __init__(self, units=30, activation='relu', **kwargs):
5         super().__init__(**kwargs)
6         self.norm_layer_wide = tf.keras.layers.Normalization()
7         self.norm_layer_deep = tf.keras.layers.Normalization()
8         self.hidden1 = tf.keras.layers.Dense(units, activation=
9             activation)
10        self.hidden2 = tf.keras.layers.Dense(units, activation=
11            activation)
12
13        self.main_output = tf.keras.layers.Dense(1)
14        self.aux_output = tf.keras.layers.Dense(1)
15
16    def call(self, inputs):
17        input_wide, input_deep = inputs
18        norm_wide = self.norm_layer_wide(input_wide)
19        norm_deep = self.norm_layer_deep(input_deep)
20        hidden1 = self.hidden1(norm_deep)
21        hidden2 = self.hidden2(hidden1)
22        concat = tf.keras.layers.concatenate([norm_wide,
23            hidden2])

```

```

20         output      = self.main_output(concat)
21         aux_output   = self.aux_output(hidden2)
22         return output, aux_output
23
24 model = WideAndDeepModel(30, name="mon-modele")
25 model.compile(loss="mse",
26               optimizer=tf.keras.optimizers.Adam(1e-2),
27               metrics=["RootMeanSquaredError"])
    
```

Couche BatchNormalization

La couche BatchNormalization apprend la moyenne et l'écart-type des données d'entraînement. Elle normalise chaque entrée de la couche; les paramètres ne sont pas affectés par la descente de gradient.

Listing 3.3 – Utilisation de la couche BatchNormalization

```

1  # Exemple type avec couche de normalisation large
2  aux = tf.keras.layers.Normalization()
3  aux.adapt(X_train)
4  model_layer_wide = aux
5  model_layer_deep = tf.keras.layers.Normalization()
6  model_layer_deep.adapt(X_train)
    
```

Callbacks : EarlyStopping et ModelCheckpoint

Listing 3.4 – Callbacks Keras – EarlyStopping et ModelCheckpoint

```

1  import tensorflow as tf
2
3  # Callback personnalisée
4  class PrintValTrainRatioCallback(tf.keras.callbacks.Callback):
5      def on_epoch_end(self, epoch, logs):
6          ratio = logs["val_loss"] / logs["loss"]
7          print(f"Epoch {epoch}, val/train = {ratio:.3f}")
8
9  # EarlyStopping : interrompt si pas d'amélioration pendant 'patience'
10 # et restaure les meilleurs poids
11 callback_es = tf.keras.callbacks.EarlyStopping(
12     patience=5,
13     restore_best_weights=True
14 )
15
16 # ModelCheckpoint : sauvegarde le meilleur modèle
17 callback_mc = tf.keras.callbacks.ModelCheckpoint(
18     "best_model",
19     save_best_only=True
20 )
21
22 history = model.fit(
23     X_train, y_train,
24     validation_data=(X_val, y_val),
    
```

```
25     epochs=90,  
26     callbacks=[callback_es, callback_mc, PrintValTrainRatioCallback  
27               (]  
28 )  
29 # Restauration du meilleur modele  
30 model = tf.keras.models.load_model("best_model")
```

Chapitre 4

Analyse du moteur DC à aimant permanent

Dans le chapitre quatre, nous nous tournons vers le moteur EG-530AD-9B de Mabuchi Motor. C'est un moteur que j'ai récupéré dans une ancienne radio Phillips, qui servait à contrôler la rotation des disquettes. Nous verrons ses caractéristiques : courant de démarrage, couple de démarrage, tension nominale, ou la vitesse de rotation nominale. Les paramètres électriques (K_t , R_a , L_a) seront dérivés à partir des spécifications constructeur.

Chapitre 5

Conception du pont en H de commande du moteur avec Arduino

Le chapitre cinq explore la conception d'un pont en H pour commander le moteur EG-530AD-9B. En effet, nous ne pouvons pas alimenter directement le moteur DC par le microcontrôleur ; cela lui serait fatal. Afin d'éviter de gâcher le cerveau des opérations, nous séparons le circuit de commande et le circuit d'alimentation du moteur ; le driver est la voie à suivre. Le circuit utilise quatre MOSFETs (Si7336ADP/Si7137DP).

Chapitre 6

Présentation des résultats des tests du système

Dans le sixième chapitre, nous présenterons tous les résultats obtenus des chapitres précédents entre autres : les performances du modèle d'apprentissage automatique ainsi que celles de sa version optimisée, le fonctionnement du modèle sur Arduino et les constats sur la précision du système en général. Ce chapitre se terminera par la conception du système d'alimentation de l'ensemble Arduino, pont en H et moteur DC, afin d'avoir un produit autonome qui ne dépend plus de l'alimentation de l'ordinateur.

Chapitre 7

Contraintes et possibilité d'étendre le projet

Le septième chapitre est un chapitre qui fixe les bases pour des prochaines investigations. Nous présentons les contraintes et difficultés rencontrées lors de la réalisation du projet ; nous parlons aussi des pistes pour étendre le projet à des applications plus robustes et exigeantes.

Chapitre 8

Conclusion

Enfin au chapitre huit, c'est la conclusion ; nous évaluons notre parcours pour voir si nos objectifs de départ ont été ou non atteints.

Annexe A

Modules Python utilisés

Les principaux modules Python utilisés dans ce projet sont les suivants :

- `tensorflow` et `keras` : construction et entraînement du réseau de neurones.
- `librosa` : acquisition et analyse audio (extraction de caractéristiques Mel).
- `sklearn` : utilitaires de prétraitement et d'évaluation.
- `matplotlib` : visualisation des données et des courbes d'apprentissage.
- `numpy` : calcul numérique et manipulation de tableaux.

Annexe B

Codes du projet

Les codes complets (Python et Arduino) sont disponibles sur le dépôt GitHub à l'adresse : <https://github.com/nobohie/prog-python>.

Bibliographie

- [1] J. D. Glover, M. S. Sarma, and T. J. Overbye, *Power System Analysis and Design*. Cengage Learning, 5th ed., 2012.
- [2] C. Bingane, M. F. Anjos, and S. Le Digabel, “Tight-and-cheap conic relaxation for the AC optimal power flow problem,” *IEEE Transactions on Power Systems*, vol. 33, no. 6, pp. 7181–7188, 2018.
- [3] C. Bingane, M. F. Anjos, and S. Le Digabel, “Tight-and-cheap conic relaxation for the optimal reactive power dispatch problem,” *IEEE Transactions on Power Systems*, vol. 34, no. 6, pp. 4684–4693, 2019.
- [4] C. Bingane, *Application de l’optimisation conique au problème d’écoulement de puissance optimal*. PhD thesis, Polytechnique Montréal, 2019.
- [5] C. Bingane, M. F. Anjos, and S. Le Digabel, “CONICOPF : Conic relaxations for AC optimal power flow computations,” in *2021 IEEE Power & Energy Society General Meeting (PESGM)*, pp. 1–5, IEEE, 2021.
- [6] C. Bingane, M. F. Anjos, and S. Le Digabel, “CONICOPF : Conic relaxations of the optimal power flow problem.” <https://github.com/cbingane/conicopf>, March 2019.